

SE 2226 Software Quality Assurance and Testing

Test Plan

Yemeksepeti Web and Mobile Application Testing

Prepared by

Atakan Sezginer – 24070006126

Batuhan Yerebasmaz – 23070006029

Ege Çınar – 24070006020

Osman Şahin Güler – 24070006009

İsmet Saygın Koç – 23070006038

19.04.2026

1. Context of Testing

1.1 Project / Test Levels / Test Types

This project focuses on testing the Yemeksepeti application, which is available as both a web application and a mobile application. The purpose of testing is to evaluate whether the system works correctly from the user's perspective and satisfies common e-commerce application expectations.

Testing will mainly be performed at the system test level, where the complete application is tested as a whole. The main test types included in this project are Functional Testing and User Interface (UI) Testing. The testing approach will follow black-box testing methods since the source code of the application is not available.

1.2 Test Items

The test items covered in this project include the Yemeksepeti Web Application and the Yemeksepeti Mobile Application. These applications allow users to search for restaurants, view menus, add food items to a cart, and complete an order.

Main components to be tested include:

- User account features (login and registration)
- Restaurant browsing and search functionality
- Viewing restaurant menus
- Cart functionality (adding items to the cart)
- Cart functionality (removing items from the cart)
- Cart functionality (changing item quantity)
- Checkout process

The test items represent the main user functionalities that are commonly used in online food ordering systems.

1.3 Test Scope

The scope of testing includes the main features of the Yemeksepeti application that affect the user experience and ordering process.

Features to be tested:

- User registration functionality
- User login functionality
- Searching restaurants by name or category
- Viewing restaurant menus

- Adding items to the cart
- Removing items from the cart
- Changing item quantity in the cart
- Entering delivery address
- Completing the checkout process
- Order confirmation behavior

Features not included in testing:

- Performance testing (load testing and stress testing)
- Security testing
- Database testing
- Internal API testing
- Unit testing

These features are excluded because the project focuses on black-box testing and the source code and infrastructure are not available.

1.4 Test Basis

The test cases will be designed based on the observable functionality of the Yemeksepeti application and general requirements of e-commerce systems. Testing will be based on user interactions with the interface and expected system behavior.

Test design techniques such as equivalence partitioning, boundary value analysis, use case testing, and decision table methods will be used to generate test cases. The expected behavior of the system will be determined according to common usability expectations and typical online ordering workflows.

2. Assumptions and Constraints

Testing of the Yemeksepeti application is conducted under certain assumptions and limitations due to restricted access to internal system components.

The testing process is based on black-box testing methods, meaning the internal source code, database structure, and system architecture are not accessible. Therefore, testing activities focus only on observable system behavior through the user interface.

The following constraints apply:

- Performance testing such as load and stress testing is excluded due to lack of access to system infrastructure.
- Security testing is not included within the scope of this project.

- Real payment transactions will not be performed; only the observable steps of the checkout process will be tested.
- Test results may vary depending on device type, browser version, mobile operating system, and internet connection.
- Mobile testing requires access to a physical device or emulator environment.

It is assumed that the publicly available features of the application represent the intended functionality of the system.

Possible risks include application updates or tool compatibility issues that may affect automated tests. In such cases, test scripts will be updated and manual testing will be performed if necessary.

3. Test Strategy

3.1 General

The testing strategy for the Yemeksepeti application focuses on evaluating the system from an end-user perspective using black-box testing techniques. Testing activities aim to verify that the main functionalities operate correctly and consistently across both web and mobile platforms.

Test cases will be designed based on user interactions and expected system behavior using equivalence partitioning, boundary value analysis, use case testing, and decision table methods.

Both manual and automated testing approaches will be used. Selenium IDE will be used for web test automation and Maestro will be used for mobile test automation. Testing will focus on core workflows such as account access, restaurant browsing, cart management, and order completion.

3.2 Test Levels

Testing will mainly be conducted at the system testing level, where the application is evaluated as a complete system from the end-user perspective across both web and mobile platforms.

Acceptance-level considerations are included since testing focuses on common user workflows such as login, restaurant search, cart operations, and order completion.

Unit and integration testing are not included because the source code and internal system structure are not accessible.

3.3 Test Types

The main test type used in this project is Functional Testing, which verifies whether the application behaves according to expected user interactions and system requirements.

User Interface (UI) Testing will also be performed to ensure that the visual components and navigation of the application function correctly on both web and mobile platforms.

Regression Testing will be performed by re-executing existing test cases to ensure previously working features continue to function correctly.

3.4 Test Deliverables

The following test deliverables will be produced during the testing process:

- Test Plan: Document describing the testing scope, strategy, and approach.
- Test Case Specifications: Detailed descriptions of test scenarios for web and mobile applications.
- Automated Test Scripts: Selenium IDE scripts for web testing and Maestro scripts for mobile testing.
- Incident Reports: Documentation of any defects or unexpected behaviors observed during testing.
- Test Status Reports: Summary of test execution progress and results.
- Test Completion Report: Final summary of testing activities and overall results.

These deliverables provide documentation of the testing process and help track the quality and reliability of the application. Identified defects will be categorized based on severity levels such as minor, major, and critical.

3.5 Test Design Techniques

Test cases will be designed using black-box testing techniques to evaluate the functionality of the application.

Equivalence Partitioning will be used to divide input data into valid and invalid groups in order to reduce the number of test cases while maintaining effective coverage.

Boundary Value Analysis will be applied to test input values at the limits of acceptable ranges, such as minimum and maximum values for quantities or form inputs.

Use Case Testing will be used to evaluate complete user workflows such as logging in, searching for restaurants, adding items to the cart, and completing an order.

Decision Table technique will be applied where different combinations of conditions lead to different system outcomes, such as login validation or checkout scenarios.

3.6 Entry and Exit Criteria

Entry Criteria

Testing will begin when the test plan is completed and approved, the test environment is ready, and the necessary tools such as Selenium IDE and Maestro are installed and configured. Test accounts required for login and ordering processes must also be available.

Exit Criteria

Testing will be considered complete when all planned test cases have been executed and the main functionalities such as login, restaurant search, cart operations, and checkout process have been tested. Any critical issues identified during testing will be documented.

Testing may be paused if critical application functions become unavailable. Testing will continue once the system becomes accessible again.

3.7 Test Completion Criteria

Testing will be considered complete when all planned test cases for both web and mobile platforms have been executed and the main user workflows such as login, restaurant browsing, cart operations, and checkout process have been tested. Test results must be documented and reviewed to ensure consistency with expected system behavior. Testing will also be considered complete when the majority of test cases have been executed successfully and no critical issues remain unresolved.

3.8 Metrics to be Collected

During testing, several metrics will be collected to evaluate the effectiveness of the test process. These include the total number of test cases executed, the number of passed and failed test cases, and the number of defects identified. These metrics help assess the quality of the application and the success of the testing activities.

3.9 Test Data Requirements

Testing requires sample user data such as email addresses and passwords for login and registration scenarios. Address information is needed to simulate the checkout process. Various input values will be used to test search functionality and cart operations. Test data will be selected to represent realistic user behavior while avoiding the use of sensitive personal information.

3.10 Test Environment Requirements

The testing environment includes both hardware and software components required to perform web and mobile testing. A personal computer will be used to create and execute automated test scripts. Google Chrome browser will be used for web application testing with Selenium IDE. Mobile testing will be conducted using a physical Android or iOS device with Maestro installed. A stable internet connection is required to access the application and perform testing activities. IntelliJ IDEA may optionally be used when test scripts need to be edited or converted to code.

3.11 Testing Activities and Estimates

The testing process includes several activities required to design, implement, and execute test cases.

- Test Planning: defining scope, strategy, and tools (1 day)
- Test Environment Setup: installing and configuring Selenium IDE and Maestro (1 day)

- Test Design: creating test cases using equivalence partitioning, boundary value analysis, use case testing, and decision tables (2–3 days)
- Test Implementation: creating and recording automated test scripts (3 days)
- Test Execution: running manual and automated tests (2 days)
- Incident Reporting: documenting defects identified during testing (performed throughout the testing process)

These estimates are approximate and may vary depending on test complexity and team coordination.

4. Staffing

4.1 General

The testing activities will be performed by a team of five members with basic knowledge of software testing and quality assurance concepts. Team members will collaborate in designing test cases, executing tests, and documenting results. Each member will contribute to both manual and automated testing activities.

4.2 Roles and Responsibilities

Test Lead – Osman Şahin Güler

Responsible for overall test planning, coordination of testing activities, and final review of test documents. The Test Lead ensures that the testing process follows the defined strategy and that all required deliverables are completed.

Web Testers – Atakan Sezginer, İsmet Saygın Koç

Responsible for designing, creating, and executing test cases for the web application. They will develop and run automated test scripts using Selenium IDE and document any defects identified during testing.

Mobile Testers – Batuhan Yerebasmaz, Ege Çınar

Responsible for designing, creating, and executing test cases for the mobile application. They will create automated test scripts using Maestro and document any issues observed during testing.

All team members will support each other when necessary and collaborate throughout the testing process.

4.3 Schedule

Testing activities will be conducted throughout the semester following the planned testing process. The activities include test planning, test design, test implementation, and test execution. All testing activities and documentation will be completed before the project presentation scheduled for May 4.

Testing Activity	Estimated Time
Test Planning	1 day
Test Environment Setup	1 day
Test Design	2–3 days
Test Implementation	3 days
Test Execution	2 days
Incident Reporting	Throughout testing